

Claves Primarias

Contents

1	Claves primarias naturales vs. claves primarias sintéticas (generadas)	1
1.1	1. Claves primarias naturales**	1
1.2	2. Claves primarias sintéticas (generadas)**	2
2	Estrategias para generar y gestionar claves sintéticas de forma adecuada	4
2.1	1. Validar unicidad en los campos clave del dominio antes de la inserción	4
2.2	2. Uso de UUIDs como claves primarias	4
2.3	3. Combinación de claves sintéticas y naturales	5
2.4	4. Control de duplicados a nivel de servicio	6
2.5	Conclusión	6

1 Claves primarias naturales vs. claves primarias sintéticas (generadas)

Una clave primaria es un identificador único para cada registro de una tabla en una base de datos. Existen dos tipos principales de claves primarias: **naturales** y **sintéticas**. A continuación, compararemos las **ventajas** y **desventajas** de cada una y exploraremos si existen formas de mitigar sus inconvenientes.

1.1 1. Claves primarias naturales**

Las **claves primarias naturales** son campos de datos que ya existen en la entidad y que tienen un valor único. Por ejemplo, en una tabla de usuarios, una clave primaria natural podría ser un número de identificación personal (como un DNI) o un email.

Ventajas:

- **Semántica significativa:** Las claves naturales suelen tener un significado intrínseco en el contexto de la entidad. Por ejemplo, un número de identificación único o un código de producto ya es relevante para el dominio de negocio.
- **Eficiencia de espacio:** No es necesario añadir una nueva columna para generar un identificador, ya que se reutiliza un valor existente.
- **Unicidad asegurada:** Si el campo ya está garantizado como único (por ejemplo, un número de seguro social o un código de producto), no hay necesidad de generar una clave adicional,

lo que simplifica el diseño de la base de datos.

Inconvenientes:

- **Cambio de valor:** En algunos casos, las claves naturales no son realmente inmutables. Por ejemplo, un email puede cambiar si el usuario actualiza su información de contacto, lo cual crea un problema, ya que la clave primaria no debería cambiar.
- **Tamaño y eficiencia:** Si la clave natural es grande (como un número de identificación alfanumérico o una combinación compleja de campos), esto puede afectar el rendimiento en consultas y en las uniones de tablas (joins) al manejar claves más largas.
- **Problemas de internacionalización:** Algunas claves naturales pueden no ser universales, por ejemplo, un número de seguridad social o un identificador regional que puede no aplicarse fuera de ciertos países o contextos.

Formas de compensar los inconvenientes:

- **Elección de una clave natural robusta:** Utilizar una clave que sea realmente inmutable, como un **código de producto estándar internacional** o un identificador que no cambie a lo largo del tiempo, puede reducir el riesgo de necesitar cambios.
- **Utilizar índices secundarios:** Se puede optar por claves primarias naturales para mantener la unicidad semántica y usar **claves sintéticas adicionales** como identificadores internos para optimizar las búsquedas y las relaciones.

—

1.2 2. Claves primarias sintéticas (generadas)**

Las **claves primarias sintéticas** son valores generados por el sistema, típicamente sin ningún significado en el dominio del negocio. Un ejemplo común es un campo de tipo **auto-increment** (en bases de datos SQL) o un **UUID**.

Ventajas:

- **Inmutabilidad garantizada:** Las claves generadas son inherentemente inmutables, ya que no dependen de datos que puedan cambiar. Esto es ideal para mantener la integridad referencial.
- **Tamaño controlado:** Las claves sintéticas suelen ser pequeñas y eficientes en términos de almacenamiento y de rendimiento en consultas y uniones de tablas, especialmente cuando se utilizan enteros autoincrementados.
- **Simplicidad en la relación de tablas:** Las claves sintéticas evitan problemas de compatibilidad entre diferentes tipos de datos o sistemas al ser utilizadas como claves externas (foreign keys) en otras tablas.

Inconvenientes:

- **Falta de significado:** Las claves sintéticas no tienen ningún significado en el negocio. Esto puede dificultar la depuración o el seguimiento, ya que los identificadores no proporcionan información directa sobre el registro.

- **Confusión en integraciones:** En sistemas que requieren integración con otros sistemas o bases de datos, las claves sintéticas pueden ser más difíciles de utilizar, ya que requieren mapeo adicional para sincronizar los datos.
- **Colisiones (en caso de UUIDs):** Aunque la probabilidad es baja, en el caso de utilizar **UUIDs** para claves primarias, existe la posibilidad (aunque extremadamente remota) de que ocurran colisiones en los valores generados.

Formas de compensar los inconvenientes:

- **Añadir un índice único en un campo significativo:** Aunque se use una clave sintética, se puede agregar un índice único en un campo natural que tenga relevancia en el negocio (como un número de identificación o código de producto) para simplificar búsquedas y relaciones.
- **Utilizar un mapeo o etiqueta en la interfaz de usuario:** Si se necesita que los usuarios vean algo significativo, se puede presentar en la interfaz un campo más natural (como el email o un código) para facilitar el trabajo humano, mientras se usa la clave sintética en el backend.
- **Generación de UUIDs en sistemas distribuidos:** Si se trabaja en un sistema distribuido o con múltiples nodos, el uso de **UUID** en lugar de enteros autoincrementados puede evitar problemas de colisión de claves y facilitar la replicación de datos.

Resumen de la comparación:**

Característica	Clave Primaria Natural	Clave Primaria Sintética
Significado	Tiene significado en el dominio del negocio	No tiene significado en el dominio
Inmutabilidad	Puede cambiar en algunos casos	Siempre inmutable
Tamaño y eficiencia	Puede ser grande y afectar el rendimiento	Generalmente pequeña y más eficiente
Diseño y mantenimiento	Más difícil si los datos cambian	Más sencillo, no requiere cambios de clave
Complejidad en relaciones	Puede ser más complejo en relaciones entre tablas	Relativamente sencillo en relaciones entre tablas
Uso en sistemas distribuidos	Puede causar problemas de integración	Ideal para sistemas distribuidos (especialmente UUIDs)

Conclusión:

- Las **claves naturales** son útiles cuando se dispone de un identificador único en el dominio de negocio, pero hay que asegurarse de que sean inmutables y eficientes.
- Las **claves sintéticas** son generalmente preferibles en la mayoría de los casos, debido a su simplicidad, eficiencia y facilidad para mantener la integridad de los datos. No obstante, la falta de significado puede ser compensada añadiendo campos naturales con índices únicos y mostrando información relevante en la interfaz de usuario.

En resumen, la elección depende del contexto del negocio y los requisitos específicos del sistema.

2 Estrategias para generar y gestionar claves sintéticas de forma adecuada

2.1 1. Validar unicidad en los campos clave del dominio antes de la inserción

- **Descripción:** Una buena práctica es asegurarse de que los campos naturales importantes (aquellos que tienen relevancia de negocio, como un email o el DNI) sean **únicos** antes de insertar el nuevo registro. De esta manera, aunque la clave primaria sea generada automáticamente, se puede evitar la duplicación de datos en campos clave para el negocio.
- **Implementación:**
 - En Spring Boot, esto se puede hacer fácilmente agregando **índices únicos** en las columnas relevantes en el esquema de la base de datos, o bien validando en el servicio o repositorio que el dato no exista antes de insertar un nuevo registro.
 - Ejemplo de índice único:

```
@Entity
public class Usuario {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(unique = true)    // Asegura la unicidad
    private String email;
    // otros campos...
}
```

- **Beneficio:** Esto evita que se inserten registros duplicados aunque la clave primaria sea generada automáticamente, ya que la base de datos lanzará una excepción si se intenta insertar un valor duplicado en un campo que tiene un índice único.

2.2 2. Uso de UUIDs como claves primarias

- **Descripción:** En lugar de utilizar claves autoincrementadas, se pueden utilizar **UUIDs** (Universally Unique Identifiers) como claves primarias. Los UUIDs son identificadores únicos a nivel global que pueden ser generados de manera distribuida sin riesgo de colisión.
- **Implementación en Spring Boot:**
 - Cambia el tipo de la clave primaria a 'UUID' y genera el UUID de forma automática.
 - Puedes usar la anotación '@GeneratedValue' con la estrategia 'UUID' o generar el UUID en tu código antes de guardar la entidad.

```
@Entity
public class Usuario {
    @Id
    @GeneratedValue(generator = "UUID")
    @GenericGenerator(name = "UUID", strategy = "org.hibernate.id.UUIDGenerator")
```

```

@Column(uptable = false, nullable = false)
private UUID id;

@Column(unique = true)
private String email;
// otros campos...
}

```

- **Beneficio:** Los UUIDs son prácticamente imposibles de duplicar y garantizan unicidad a nivel global, lo cual es especialmente útil en **sistemas distribuidos** o cuando varias bases de datos independientes pueden estar insertando datos simultáneamente. Esto es útil si no confías en secuencias de autoincremento locales y si se requiere integración con otros sistemas.

2.3 3. Combinación de claves sintéticas y naturales

- **Descripción:** En algunos casos, puede ser útil utilizar una clave sintética (como un autoincremento o un UUID) como clave primaria, pero también usar una **clave natural compuesta** para garantizar unicidad en los datos de negocio. Esta clave natural no sería la clave primaria, pero funcionaría como una **clave alternativa** o **clave compuesta** para evitar duplicados.
- **Implementación:** Se puede definir un índice único compuesto por varios campos naturales que son importantes en el negocio (por ejemplo, 'email' y 'número de identificación'), mientras que la clave primaria sigue siendo una clave sintética.
- **Ejemplo:**

```

@Entity
@Table(uniqueConstraints = {@UniqueConstraint(columnNames = {"email", "dni"})})
public class Usuario {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false)
    private String email;

    @Column(nullable = false)
    private String dni;
    // otros campos...
}

```

- **Beneficio:** Esto permite garantizar que no haya duplicados en campos clave del negocio, pero al mismo tiempo se mantiene una clave primaria sintética simple para mejorar la eficiencia de las consultas y las relaciones en la base de datos.

2.4 4. Control de duplicados a nivel de servicio

- **Descripción:** Además de usar mecanismos de la base de datos para controlar la unicidad, es importante implementar lógica en el **servicio** que controle si los datos ya existen antes de intentar insertarlos. Esto se puede hacer con un chequeo previo mediante consultas.
- **Implementación:** En el servicio que maneja las inserciones, antes de guardar un nuevo registro, se puede consultar si ya existe un registro con los mismos datos relevantes. Si existe, se lanza una excepción o se retorna un mensaje de error.

```
@Service
public class UsuarioService {

    @Autowired
    private UsuarioRepository usuarioRepository;

    public Usuario crearUsuario(Usuario usuario) {
        if (usuarioRepository.existsByEmail(usuario.getEmail())) {
            throw new RuntimeException("Ya existe un usuario con este email");
        }
        return usuarioRepository.save(usuario);
    }
}
```

- **Beneficio:** Este enfoque permite manejar duplicados de manera más flexible, ya que se puede devolver un mensaje adecuado al usuario o realizar alguna acción adicional antes de que ocurra la inserción. Complementa la validación en la base de datos.

2.5 Conclusión

Si bien las **claves sintéticas** como las generadas automáticamente mediante **autoincrementos** o **UUIDs** ofrecen una solución eficiente y limpia para garantizar unicidad en la clave primaria, deben ir acompañadas de **mecanismos adicionales** para evitar la inserción de datos duplicados en campos relevantes para el negocio. Esto se puede lograr mediante:

1. **Índices únicos** en campos clave de negocio.
2. **Validaciones previas** en el servicio antes de la inserción.
3. El uso de **UUIDs** si se requiere unicidad global en sistemas distribuidos.
4. **Claves compuestas** cuando se necesite más robustez en la validación de datos únicos.

Estas estrategias combinadas permiten compensar las limitaciones de la generación automática de claves sintéticas y asegurar la integridad de los datos.